

Major Project — Engineering & AI Coding Project Pack

PRD + Architecture + Rules + Phases + Design + Memory + Edge Cases + Experiment Protocol + Codex Workflow + BMAD-style adaptation

Living engineering/control documentation. Update it when real project decisions change.

PRD — Project Requirements Document

Working Project Title

Uncertainty-Aware Agent for Reliable Brain Tumor Segmentation Under Heterogeneous MRI Conditions

Mentor Direction

“Uncertainty-aware agent dynamically selects segmentation strategies, modalities, and specialized models for reliable brain tumor segmentation under heterogeneous MRI conditions.”

Goal

Build a development-backed research project for brain tumor MRI segmentation. First establish clean-image segmentation baselines, then study heterogeneous/degraded conditions, uncertainty/reliability, and finally an agentic selection layer.

Core capabilities

- Load and validate BraTS-GLI MRI data.
- Preprocess clean MRI data.
- Train/evaluate representative segmentation models.
- Visualize MRI, ground truth, and predictions.
- Calculate segmentation metrics.
- Compare models under controlled conditions.
- Introduce controlled degradation only after clean baselines.
- Estimate uncertainty/reliability.
- Dynamically select an appropriate model/strategy/modality configuration.
- Log experiments for reproducibility.

Non-goals

- Do not force LLM, RAG, NLP, Generative AI, or MCP into the project without a real use.
- Do not fabricate results.
- Do not claim U-Net or another established architecture as an original invention.
- Do not modify raw dataset files.

First demonstrable milestone

Clean BraTS-GLI → preprocessing → 2D U-Net → segmentation → Dice/IoU/precision/recall → visualization.

Architecture

Overall evolution

Clean MRI → validation → preprocessing → model benchmark → controlled heterogeneity/degradation → uncertainty estimation → agentic decision → selected segmentation strategy → reliability report.

Initial system

Raw BraTS-GLI → dataset inspector → preprocessor → slice/volume dataset → U-Net trainer → evaluator → visualization.

Final direction

MRI input → condition/quality information → uncertainty estimation → agent → select modality/preprocessing/model → inference → segmentation + uncertainty/reliability report.

Candidate models

1. U-Net
2. U-Net++
3. Attention U-Net
4. ResUNet / residual U-Net
5. U-Net with ResNet encoder
6. TransUNet
7. 3D U-Net only if data/hardware design permits.

Stack

Python 3.11, PyTorch + CUDA, MONAI, NiBabel, NumPy, pandas, scikit-learn, matplotlib, tqdm, Git/GitHub, Codex/local AI coding assistance.

Hard constraints

No Keras, TensorFlow, or MATLAB. Use PyTorch for deep learning. Preserve raw data and make experiments reproducible.

Rules

Project rules

- Follow the current Major Project and supervisor requirements.
- Include real development, not pure research.
- Mission AI alignment must be substantive.
- Novelty claims require literature support.
- Never fabricate metrics, figures, experiments, or citations.
- Maintain the supervisor meeting diary.
- Treat current mentor instructions as authoritative when they refine earlier assumptions.

Technical rules

- PyTorch only for the deep-learning implementation.
- No Keras, TensorFlow, MATLAB.
- Never modify raw dataset files.
- Record experiment configuration.
- Change one major experimental factor at a time unless studying interactions.
- Accuracy is not the sole segmentation metric.
- Never report an experiment as complete until it has actually run.

AI coding rules

AI-generated code is assistance, not evidence. Run it, test it, inspect it, understand its inputs/outputs, and keep a short viva explanation.

Agent rules

The agent must make an observable decision using defined inputs and a defined policy/objective. A fixed if/else baseline must not be presented as a learned/intelligent agent without saying so.

Phases

Phase 0 — Environment

Python 3.11.9, dedicated D:\MajorProject\venv, CUDA-enabled PyTorch, RTX 4070 verified with an actual CUDA computation.

Phase 1 — Dataset

Download BraTS-GLI training/validation, retain mapping and README, extract safely, inspect cases, verify modalities/masks, detect missing/corrupt data.

Phase 2 — Clean preprocessing

Load NIfTI, validate dimensions/orientation/metadata, normalize intensity, choose 2D/volume strategy, build reproducible data pipeline.

Phase 3 — First baseline

Implement 2D U-Net, train on GPU, save checkpoints and curves, generate prediction visualizations, calculate Dice/IoU/precision/recall.

Phase 4 — Model benchmark

Implement representative U-Net variants and other agreed architectures under controlled conditions. Never fabricate missing results.

Phase 5 — Experimental analysis

Study selected factors such as epochs, batch size, dataset fraction, augmentation, modality configuration, model complexity, training time, and inference time.

Phase 6 — Heterogeneous/degraded conditions

Only after the clean baseline is stable. Use mentor-approved degradation/noise types and severity levels. Preserve clean references.

Phase 7 — Uncertainty

Implement and evaluate a defined uncertainty/reliability mechanism.

Phase 8 — Agentic selection

Use defined condition/reliability information to select a segmentation strategy/model/modality configuration.

Phase 9 — Final evaluation

Compare fixed and dynamic strategies using segmentation quality, reliability, robustness, and compute cost.

Phase 10 — Report and viva

Convert actual evidence into the report, diagrams, tables, limitations, and viva explanations.

Design

Demo/dashboard views

1. Case selector
2. MRI modality viewer
3. Ground truth vs prediction
4. Metrics panel
5. Model comparison
6. Uncertainty/reliability view
7. Agent decision trace
8. Experiment history

Visualization

Show MRI, ground truth, prediction, quantitative metrics, selected model, and the inputs that influenced the selection.

Every figure/table should be traceable to an experiment ID and configuration.

Memory / Project State

Current state

- Project root: D:\MajorProject
- Python 3.11.9
- PyTorch 2.11.0+cu128
- CUDA available
- NVIDIA GeForce RTX 4070 Laptop GPU
- Actual CUDA computation verified
- BraTS-GLI training/validation downloads in progress

Current milestone

Dataset acquisition and environment setup.

Next milestone

Inspect extracted BraTS-GLI structure and build the dataset inspector.

Immediate next tasks

1. Finish downloads.
2. Install medical-imaging dependencies.
3. Extract safely.
4. Inspect modalities and segmentation masks.
5. Build clean preprocessing.
6. Train first U-Net.

Important decisions

- Clean-image baseline first.
- Noise/degradation later.
- Start with 2D due practical 8 GB VRAM constraints.
- Preserve raw data.
- Do not force MCP/LLM/RAG/etc.

AI workflow

ChatGPT: architecture, research framing, experiment design, review, viva.

Codex: scoped implementation/debugging.

User: run, verify, understand, document.

Edge Cases

Data

- missing modality
- missing mask
- corrupted NIfTI
- inconsistent dimensions
- unexpected orientation/spacing
- empty tumor mask
- duplicate case
- train/validation leakage
- severe class imbalance

Preprocessing

- NaN/Inf
- extreme intensity outliers
- all-zero slice
- no-tumor slice
- crop removes anatomy
- incorrect mask interpolation
- image/mask misalignment

Training

- GPU out-of-memory
- NaN loss
- exploding gradients
- validation metric stagnation
- checkpoint corruption
- slow data loading

Evaluation

- ambiguous Dice when both masks are empty
- division by zero
- wrong label
- accidental validation-data training
- unfair model comparison

Agent

- no valid candidate
- all candidates highly uncertain
- requested modality missing
- threshold boundary

- compute budget exceeded
- invalid action
- contradictory quality/uncertainty signals

Demo

- unsupported file
- oversized input
- slow inference
- missing checkpoint
- wrong device
- CPU fallback
- stale result displayed as current

Experiment Protocol

Baseline principle

Keep the dataset split, preprocessing, augmentation, optimizer, learning rate, batch size, epochs, loss, and seed controlled when comparing architectures.

Metrics

Primary: Dice, IoU/Jaccard.

Secondary: precision, recall/sensitivity, specificity where appropriate, Hausdorff distance if feasible, parameter count, training time, inference time.

Traceability

Every result should identify dataset split, model/version, configuration, checkpoint, code revision, and experiment ID.

Clean vs degraded

Do not compare conditions without a documented evaluation protocol. The exact degradation/noise types and severity levels should follow mentor guidance before final experiments.

Codex Workflow

Principle

Do not ask Codex to build the whole project in one prompt.

Every task should specify

- context
- exact files to create/change
- inputs/outputs
- constraints
- acceptance tests
- what must not change

Example task

Create `dataset_inspector.py` that uses `NiBabel` to inspect a specified BraTS-GLI directory, identify expected modality/mask files, report missing files and dimensions, never modify data, and run on one case as a test.

Milestones

- v0.1 project skeleton
- v0.2 dataset inspector
- v0.3 preprocessing
- v0.4 U-Net
- v0.5 metrics
- v0.6 visualization
- v0.7 experiment logger
- v0.8 U-Net++
- v0.9 additional models
- v1.0 clean benchmark
- v1.1 controlled degradation
- v1.2 uncertainty
- v1.3 agentic selection
- v1.4 final dashboard/report

After every task

Run tests, inspect the diff, understand the module, update `Memory.md`, and create a meaningful Git checkpoint.

BMAD-Style Adaptation

BMAD is optional. We do not need a methodology label just for appearance.

The useful idea is staged, artifact-driven work:

- PRD → what the system must accomplish
- Architecture → how it is structured
- Rules → hard constraints
- Phases → implementation sequence
- Design → visual/demo decisions
- Memory → current state and decisions
- Edge Cases → failure modes
- Experiment Protocol → scientific evaluation
- Codex Workflow → scoped AI coding tasks

Why this helps

It prevents Codex from inventing architecture, preserves context across tools, reduces uncontrolled rewrites, improves reproducibility, and creates useful project documentation.

Do not over-process

These are living documents. Update them when real decisions change; do not spend weeks writing process documents instead of building the system.

PROJECT_SPEC.md

Project

Uncertainty-Aware Agent for Reliable Brain Tumor Segmentation Under Heterogeneous MRI Conditions

Current goal

Clean BraTS-GLI segmentation baseline → heterogeneous/degraded conditions → uncertainty → agentic dynamic selection.

Hard constraints

PyTorch; no Keras; no TensorFlow; no MATLAB; preserve raw data; no fabricated results; no forced AI technologies.

Environment

Python 3.11.9; PyTorch 2.11.0+cu128; CUDA available; NVIDIA RTX 4070 Laptop GPU; root D:\MajorProject.

Current dataset

BraTS-GLI training and validation data, mapping spreadsheet, README.

Current phase

Dataset acquisition / environment setup.

Immediate next task

Inspect extracted BraTS-GLI data and create a dataset inspector.

First demo

Clean MRI → preprocessing → 2D U-Net → segmentation → Dice/IoU/precision/recall → visualization.

Later

Model benchmark → controlled degradation → uncertainty → agentic modality/strategy/model selection.

AI coding

ChatGPT for architecture/research/review; Codex for scoped implementation; user runs, verifies, understands, and records.